Attention Enhancement for Long-Context Large Language Models Based on Hamiltonian-Path Virtual-Edge Structures

Yunshao, Lingxing Zhishi

Technical Paper • Academic Exchange Edition 1.0

Attention Enhancement for Long-Context Large Language Models Based on Hamiltonian-Path Virtual-Edge Structures

Technical Paper • Academic Exchange Edition 1.0

Abstract

Modern large language models face the dual challenges of attention sparsification and long-range information propagation in long-context scenarios. This paper proposes an attention enhancement framework based on the edge-structure analysis of Hamiltonian paths, decomposing the model's attention graph into four edge types: virtual edges, real edges, deficient edges, and curved edges, corresponding respectively to four operations: long-range shortcuts, backbone protection, blind-spot diagnosis, and local dead-end breakthrough. Among these, the virtual-edge mechanism constructs direct shortcuts between distant semantically similar units, allowing information to propagate without layer-by-layer attenuation along the sequence, thereby providing a structural basis for long-range attention and sparse attention. Experiments are conducted on Qwen2.5-0.5B and Qwen2.5-7B. The results show that structural criteria comprehensively outperform norm criteria in sparse attention (perplexity reduced by up to 38.3%); pruning the backbone attention heads identified by the structural analysis causes the perplexity to collapse by factors of 136 to more than 5000, whereas pruning redundant heads has negligible impact. This paper provides complete mathematical formulations and a reference

implementation; all thresholds are tunable parameters and can be directly ported for reproduction.

Keywords: Hamiltonian path; attention mechanism; sparse attention; model pruning; long context; structural analysis

1. Introduction

The attention mechanism of large language models incurs significant computational and memory overhead as sequence length grows. Mainstream solutions include sliding-window local attention, sparse attention, and KV-cache compression. However, most of these methods rely on simple criteria such as norms or distances to select key positions, lacking a systematic understanding of the internal information structure of the model.

This paper adopts the perspective of the edge structure of Hamiltonian paths, treating the attention graph as a structured graph and partitioning all connections into four categories according to the existence and role of edges. This framework simultaneously addresses four engineering questions: which connections must be preserved (the real-edge backbone); which missing connections cause information blind spots (deficient edges); which long-range shortcuts can be proactively constructed (virtual edges); and how to break out of local optima (curved edges). The entire method uses only generic graph-theoretic and linear-algebraic notation, all thresholds are tunable parameters, and no specific constant assumptions are made. The mathematical formulations and reference implementation are given in Sections 7 and 8.

2. Problem Definition and the Four-Edge Framework

Let A be the adjacency matrix obtained by thresholding the attention matrix of N tokens, where $A_{ij} = 1$ if and only if the attention weight w_{ij} is greater than the threshold θ_e, with a causal mask applied so that only historical positions are retained. Four edge types are defined on this graph:

First, real edges: existing strong connections that constitute the backbone of information flow in the model. Experiments show that pruning 10% of backbone attention heads raises perplexity by a factor of 136 (7B model), whereas pruning the same proportion of redundant heads raises it by only 1.06 to 1.38 times. Real edges are untouchable lifelines and serve as constraints for pruning and quantization operations.

Second, virtual edges: direct shortcuts between units that are far apart but semantically or phase-similar. Virtual edges allow information to be injected directly without passing token by token through intermediate tokens, breaking the sequence-level propagation bottleneck. The virtual-edge mechanism in this paper is consistent with the design of indexers in mainstream sparse-attention mechanisms: the indexer scores by the product of semantic inner products and rotational-position-encoding phase alignment, establishing high-weight shortcuts at positions where distant pairs become phase-aligned.

Third, deficient edges: connections that should exist but are missing, corresponding to structural blind spots in the attention graph and high-incidence regions of hallucination. By locating deficient edges layer by layer and head by head, the model's information blind spots can be predicted at the structural level.

Fourth, curved edges: detour strategies that deliberately select neighbors that are farther away in the low-dimensional embedding, used to break out of local dead ends caused by greedy nearest-neighbor selection.

3. Core Method: Virtual Edges and Long-Range Shortcuts

3.1 Scoring and edge construction

Virtual-edge selection relies on similarity scoring. Let query q and key k be processed by rotary position encoding; the score is defined as the inner product after encoding (see Equation 7.4 in Section 7). Multi-frequency rotary position encoding causes position pairs to become phase-aligned at specific periodic distances, so that keys that are distant but phase-similar receive high scores, enabling long-range direct jumps.

Nearby neighbors are handled by an independent sliding-window branch, and the indexer performs only long-distance selection, forming a near-far division of labor.

In scenarios without rotary position encoding, a common-neighbor proxy score can be used as a structural criterion instead: compute the inner product of query and key vectors averaged across attention heads, then select the top-k global keys by score.

3.2 Comparison of structural criteria and norm criteria

On the local Qwen2.5-0.5B model, sparse-attention perplexity is compared between the common-neighbor method (structural criterion) and norm top-k (norm criterion). Results are shown in Table 1 (lower PPL is better).

Table 1. Perplexity comparison of structural and norm criteria

Configuration	Norm criterion	Structural criterion	Reduction
Length 4K, keep 10%	3.547	2.188	38.3%
Length 8K, keep 10%	1.245	1.087	12.7%
Length 16K, keep 5%	1.123	1.044	7.0%
Length 16K, keep 2.5%	3.011	2.036	32.4%
Length 32K, keep 2.5%	1.072	1.032	3.7%

The structural criterion wins in all five configurations; the higher the sparsity, the more significant the advantage of the structural criterion over the norm criterion.

3.3 Structural evidence for long-range shortcuts

The structural criterion for virtual-edge long-range shortcuts has been validated on the local model: the common-neighbor method (structural criterion) outperforms the norm criterion in all five sparse configurations (see Table 1), with the advantage growing as sparsity increases. End-to-end long-range capability validation on ultra-long contexts must be reproduced on the target model; this paper does not introduce third-party model measurement data.

3.4 Scale scaling of virtual-edge disconnection

In geometric-body edge-disconnection experiments, the number of virtual-edge disconnections grows linearly with scale, exhibiting a strict O(N) scaling: 407,008 disconnections at 10^7 nodes and 4,069,228 at 10^8 nodes, with the ratio exactly equal to 10; the disconnection density converges to 4.07% at large scales. All disconnection endpoints are low-degree nodes (average degree 3.01 versus a full-graph mean of 6.00), and none involves hub nodes, indicating that virtual-edge deficiency is localized to structurally sparse regions. A single-pass greedy run at 10^8 scale takes only 152.5 seconds.

4. Real Edges: Backbone Identification and Pruning Guidance

4.1 Structural concentration metric

After thresholding the attention matrix into a graph, the product of in-degree and out-degree of a node measures its importance in information aggregation. The structural concentration is defined as the ratio of the maximum importance to the mean importance minus one (see Equation 7.1 in Section 7). High concentration indicates that a few key tokens aggregate most of the information, corresponding to the real-edge backbone; low concentration indicates uniform information distribution, corresponding to prunable redundancy. Nodes or layers are classified into three states by concentration: backbone above the upper threshold, redundant between the two thresholds, and uniform below the lower threshold (see Equation 7.2).

4.2 Pruning experiments

On the 7B model: pruning 10% of backbone attention heads raises perplexity by a factor of 136.59; pruning 10% of redundant heads raises it by only 1.88. On the 0.5B model (336 attention heads, full perplexity 19.05): pruning 5% of heads by the structural criterion raises perplexity by 19.3×; pruning 10% by 5053×; pruning 20% by 43096×; whereas random pruning of 30% raises it by only 2.64×. The structural criterion is far superior to random and entropy criteria in identifying lifelines, and the behavioral pattern of the 0.5B and 7B models is scale-invariant.

4.3 KV-cache budget allocation

The cache retention ratio is adaptively allocated by the structural concentration of each layer: layers with high concentration retain more (see Equation 7.3). On Qwen2.5-0.5B, structure-guided allocation reduces perplexity relative to uniform allocation at all retention ratios, with the optimum at 75% retention (a 7.9% reduction); the deliberately reversed control group (retaining more in low-concentration layers) instead increases perplexity by 27.2%, confirming that backbone protection is not a psychological artifact.

5. Deficient Edges: Structural Blind Spots and Hallucination Diagnosis

On the 0.5B model, the three-state distribution is 93.15% backbone, 6.55% redundant, and 0.30% unresolvable. Notably, the three-state proportions invert between the 0.5B and 7B models (7B backbone 6.76%), but the relative relation between backbone and redundancy is scale-invariant: regardless of scale, pruning the backbone causes collapse while pruning redundancy is safe.

The criterion for a deficient edge is a token pair with high structural or semantic correlation but an attention weight below the graph-construction threshold (see the candidate criterion in Equation 7.2). Such candidate pairs are high-incidence blind spots for hallucination and can be located layer by layer and head by head. In the degree-completion experiment (33 nodes), after raising the leaf-node degree from 1 to 2, a strict Hamiltonian path exists and achieves 100% coverage with a real-edge ratio of 1.000, verifying that deficient-edge completion is a necessary operation for structural completeness.

6. Curved Edges: Low-Dimensional Detours to Break Local Dead Ends

After embedding nodes in two-dimensional coordinates, the greedy selection of the nearest neighbor in low-dimensional distance achieves only 0.067

coverage on BA graphs (1000 nodes) and 0.049 (5000 nodes); switching to the farthest neighbor in low-dimensional distance (curved edges, see Equation 7.5) raises coverage to 0.160 and 0.088, an improvement of 2.4× and 1.8×, respectively, with all paths composed of real edges (real-edge ratio remains 1.000). Curved edges correspond to escaping local-optimum traps in search, planning, reasoning chains, and optimization-escaping scenarios.

7. Core Mathematical Formulations (Complete Specification)

The following is the complete mathematical specification. All thresholds are tunable parameters, using only generic graph-theoretic and linear-algebraic notation.

Notation: A is the thresholded causal attention adjacency matrix; $N(v)$ is the neighbor set of node v; $\bar{d}$ is the average degree; $x(v)$ is the low-dimensional embedding coordinate; θe, τhi, τlo, k, β, r are tunable hyperparameters.

Equation 7.1. Structural concentration metric

In-degree and out-degree: deg_in(v) = Σu A_uv, deg_out(v) = Σu A_vu.

Information-aggregation importance: imp(v) = deg_in(v) × deg_out(v).

Structural concentration: C(G) = max_v imp(v) / ((1/n) Σv imp(v)) − 1.

Interpretation: high C(G) means a few key tokens aggregate most information, corresponding to the real-edge backbone (not prunable); low C(G) means uniform information distribution, corresponding to redundancy (prunable).

Equation 7.2. Three-state classification and deficient-edge criterion

State classification (τhi, τlo are tunable thresholds):

If Cv > τhi, then backbone (real edge, preserve);

If τlo < Cv ⩽ τhi, then redundant (compressible);

If Cv ⩽ τlo, then uniform (prunable).

Deficient-edge criterion (structural blind spots where connections should exist but do not): token pairs with high structural or semantic correlation but attention weight below the graph-construction threshold θ_e:

candidate(i,j) holds if and only if $\mathrm{sim}(i,j) > \tau_s$ and $A_{ij} < \theta_e$.

These candidate pairs are high-incidence blind spots for hallucination and can be located layer by layer and head by head.

Equation 7.3. Layer-wise KV-cache budget allocation

Normalized concentration: $\hat{c}_l = (c_l - \min c) / (\max c - \min c)$, ranging from 0 to 1.

Adaptive budget: $b_l(\text{adaptive}) = r \times (1 + \beta(1 - \hat{c}_l))$; high-concentration layers retain more.

Normalize to the total budget and clip: $b_l \leftarrow b_l \times (r L / \Sigma_l b_l)$, $b_l = \mathrm{clip}(b_l, 0.1, 1.0)$.

Control configurations: uniform allocation $b_l = r$; deliberately reversed $b_l(\text{inverse}) = r(1 + \beta \hat{c}_l)$.

Equation 7.4. Virtual-edge criterion and common-neighbor sparse attention

Score (semantics times phase alignment): $s(q_i, k_j) = \langle \mathrm{RoPE}(q_i), \mathrm{RoPE}(k_j) \rangle$.

Rotary position encoding phase (multi-frequency, periodic distance dependence): $\theta_d = 1 / (\mathrm{base}^{(2d/D)})$; $\mathrm{RoPE}(x)_d = x_d \times e^{(i p \theta_d)}$. Here p is the position; multi-frequency encoding makes position pairs phase-align at periodic resonance distances, so distant but phase-similar keys receive high scores, establishing long-range shortcuts.

Edge construction (top-k sparse): selected = $\mathrm{topk}(\{s(q_i, k_j)\}_j, k)$. Nearby neighbors are handled by an independent sliding-window branch; the indexer performs only long-distance selection.

Common-neighbor proxy score (structural criterion when RoPE is absent): $\bar{q} = (1/H_q) \Sigma_h q(i,h)$, $\bar{k}_j = (1/H_k) \Sigma_h k(j,h)$, $s_j = \langle \bar{q}, \bar{k}_j \rangle$. Again select global keys by top-k.

Equation 7.5. Curved edges (low-dimensional detour selection)

Low-dimensional distance: $d_2(v,u) = \| x(v) - x(u) \|_2$ ($x(v)$ is the low-dimensional embedding coordinate; $N(v)$ is the neighbor set).

Nearest-neighbor baseline: n* = argmin_{u∈N(v)} d2(v,u).

Curved edge (detour): n* = argmax_{u∈N(v)} d2(v,u).

Interpretation: greedy nearest-neighbor selection in low dimensions falls into local dead ends (nearest-neighbor coverage 0.067 versus detour 0.160, a 2.4× improvement); detouring equals breaking out of local optima.

8. Reference Implementation

The following Python skeleton corresponds one-to-one with the equations in Section 7 (structural concentration ↔ 7.1, three states ↔ 7.2, budget ↔ 7.3, virtual edges and common neighbors ↔ 7.4, curved edges ↔ 7.5). All variable names match the equation notation and can be directly ported.

8.1 Structural concentration metric (Equation 7.1)

```
import numpy as np

def structural_concentration(attn_matrix, edge_thresh=0.1):
    """
    Threshold the attention matrix into a graph, count how often each node is
    attended to (in-degree) and how often it attends to others (out-degree).
    Importance = in-degree * out-degree. Concentration = max importance /
    mean importance - 1. High concentration = few key tokens aggregate most
    information (real edges = backbone, not prunable); low concentration =
    uniform information distribution (redundant, prunable).
    edge_thresh: attention weight threshold; weights above it count as edges.
    """
    causal = np.tril(attn_matrix)                    # causal mask: history only
    adj = (causal > edge_thresh).astype(np.int8)     # threshold into graph
    in_deg = adj.sum(axis=0)                         # in-degree
    out_deg = adj.sum(axis=1)                        # out-degree
    imp = in_deg * out_deg                           # aggregation importance
    mean_imp = imp.mean()
    if mean_imp <= 1e-10:
        return 0.0
    return imp.max() / mean_imp - 1
```

8.2 Three-state classification (Equation 7.2)

```
def classify(concentration, hi_bound=2.0, lo_bound=1.0):
    """
    Classify nodes/layers into three states by structural concentration:
      > hi_bound : backbone (real edges, information lifeline, untouchable)
      middle     : redundant (uniformly distributed, safely compressible)
      < lo_bound : uniform (near-random, prune first)
    hi_bound / lo_bound are tunable thresholds.
```

```
    """
    if concentration > hi_bound:
        return "skeleton"    # backbone, preserve
    elif concentration > lo_bound:
        return "redundant"   # redundant, compressible
    else:
        return "uniform"     # uniform, prunable
```

8.3 Layer-wise KV-cache budget allocation (Equation 7.3)

```
def allocate_budget(ratio_avg, beta, layer_conc, mode="adaptive"):
    """
    Allocate the cache retention ratio by each layer's structural
    concentration. High-concentration layers (strong real-edge backbone
    signals) retain more; uniform layers compress more.
    ratio_avg: average retention ratio (e.g., 0.75); beta: adaptation
    strength (tunable); mode: uniform=baseline / adaptive=by concentration /
    inverse=reversed control.
    """
    n_layers = len(layer_conc)
    budgets = np.zeros(n_layers)
    cmin, cmax = layer_conc.min(), layer_conc.max()
    for li in range(n_layers):
        norm = (layer_conc[li] - cmin) / (cmax - cmin) if cmax > cmin else 0.5
        if mode == "uniform":
            budgets[li] = ratio_avg
        elif mode == "adaptive":
            budgets[li] = ratio_avg * (1 + beta * (1 - norm))
        elif mode == "inverse":
            budgets[li] = ratio_avg * (1 + beta * norm)
    total = budgets.sum()
    budgets = budgets * (ratio_avg * n_layers / total)
    return np.clip(budgets, 0.1, 1.0)
```

Measured (Qwen2.5-0.5B, 216 tokens, 24 layers, full perplexity 19.05): at 90% retention, structure-guided 19.45 vs. uniform 20.53 (5.3% lower); 80%: 20.60 vs. 21.66 (4.9% lower); 75%: 21.42 vs. 23.25 (7.9% lower); 70%: 22.10 vs. 23.89 (7.5% lower); 60%: 27.24 vs. 28.13 (3.2% lower); 50%: 39.99 vs. 41.20 (2.9% lower). Control (75% retention, deliberately reversed allocation): perplexity 24.24, 27.2% higher than uniform allocation, confirming that backbone protection is not a psychological artifact.

8.4 Common-neighbor sparse attention (Equation 7.4, core of virtual edges)

```
import torch
```

```python
def co_neighbor_global_attn(qb, k, n_global):
    """
    Common-neighbor method for selecting global attention keys: for each query
    block, select the keys with the most shared attention with it for global
    attention. Token pairs with high shared attention = strongly correlated
    (information-relevant), preserve; low shared attention = weakly
    correlated, can be skipped (sparsification).
    qb: (H_q, D) block-level query representation; k: (N, D) cross-head key
    mean; n_global: number of global keys to retain.
    """
    with torch.no_grad():
        probe = qb.float().mean(dim=1)      # (H_q, D) block representative query
        probe_avg = probe.mean(dim=0)       # (D,) cross-head average
        k_avg = k.float().mean(dim=0)       # (N, D) cross-key-head average
        score = k_avg @ probe_avg           # (N,) shared-attention proxy score
        n_eff = min(n_global, k_avg.shape[0])
        idx = torch.topk(score, n_eff).indices
        return torch.sort(idx).values       # selected global key indices (ascending)
```

8.5 Curved edges (Equation 7.5, low-dimensional detour selection)

```python
def curved_edge_next(embed2d, current, neighbors, ratio=1.5):
    """
    Curved-edge method for next-step selection: under the low-dimensional
    embedding, select the neighbor with the largest low-dimensional distance
    (detour). Greedy nearest-neighbor selection easily falls into local dead
    ends; selecting low-dimensional distant neighbors bends the path in the
    low-dimensional view, covering more nodes and breaking local dead ends.
    embed2d: node low-dimensional embedding coordinates (N, 2); current:
    current node; neighbors: candidate neighbor list; ratio: detour strength
    (tunable, >1 means selecting distant neighbors).
    """
    cx, cy = embed2d[current]
    def dist(n):
        nx, ny = embed2d[n]
        return ((nx - cx) ** 2 + (ny - cy) ** 2) ** 0.5
    dists = [(n, dist(n)) for n in neighbors]
    dists.sort(key=lambda x: x[1], reverse=True)   # select farthest
    k = max(1, int(len(dists) / ratio))
    return dists[:k]                               # detour candidates
```

9. Engineering Deployment Suggestions

Stage 1 (weeks 1–2): real-edge and deficient-edge diagnosis integration. Run structural analysis on the target model, output the three-state distribution of each layer; locate deficient-edge blind spots and real-edge backbone distributions; produce backbone and redundancy distribution maps and compression-ratio-vs-perplexity curves.

Stage 2 (weeks 2-4): virtual-edge sparse attention integration. Use the common-neighbor method (structural criterion) to replace or augment norm top-k sparse selection; compare long-range task hit rates on the target model.

Stage 3 (weeks 4-8): curved-edge and hallucination-diagnosis applications. Use curved-edge thinking to enhance reasoning chains and search escape; implement structured hallucination diagnosis, and after blind-spot localization, verify the correlation between blind spots and actual errors in task-specific settings.

10. Discussion

The four edge types form a complementary and complete operation set: real edges protect the backbone lifeline, deficient edges locate blind spots, virtual edges construct long-range shortcuts, and curved edges break local dead ends. The method relies entirely on tunable thresholds and generic graph-theoretic notation and can be ported to any Transformer architecture.

Honest boundaries: first, itemized quantitative analysis of hallucination ablation remains to be refined; second, learned KV merging requires training-environment support; third, the 0.5B model is the primary test subject, and larger scales need reproduction on the target model; fourth, the curved-edge experiments are at the path level, and attention-layer deployment requires further validation. In addition, visual and speech perception layers are not within the scope of this graph-structural analysis, and low-level operator optimization is also outside the scope.

11. Conclusion

This paper proposes an attention enhancement framework based on the four edge structures of Hamiltonian paths and validates it with measurements across multiple model scales: structural criteria outperform norm criteria, backbone identification is scale-invariant, virtual edges support long-range shortcuts, and curved edges break local dead ends. The method provides a unified structural basis for sparse attention, KV-cache compression, pruning and quantization, and hallucination diagnosis in large language models, with the conditions for direct reproduction and engineering deployment.

Version: Academic Exchange Edition 1.0

Date: August 2026

Note: This document is for academic exchange purposes. All thresholds are tunable engineering parameters, using only generic graph-theoretic and linear-algebraic notation.

> Note: This paper is a projection of the meta-structure framework onto large language model attention. The framework spans several independent papers, each self-contained with its own evidence chain: mathematical unity, material strength, superconductivity, and fundamental field theory. See the other papers in this series.